

Affirm BNPL — Setup & How It Works

Real, legal buy-now-pay-later financing at marketplace checkout, for **real shippable goods only**. Affirm underwrites each buyer and owns the default risk; **you are paid upfront**. Off by default. Not legal/financial advice.

What this is (and isn't)

- **Is:** a checkout option where a buyer *chooses* to finance a real product. Affirm approves the buyer (or not) and sets their limit, pays you the order total upfront (minus Affirm's merchant fee), and collects the installments from the buyer. If the buyer defaults, **Affirm** handles it — not you.
- **Isn't:** a flat grant of credit to everyone, and **not** for points, store credit, "play" credit, or affiliate items. Affirm's terms and consumer-lending law prohibit financing gambling/play value — the code hard-blocks those (real USD price + tangible-goods guard).

Get Affirm keys

1. Apply for an Affirm merchant account (affirm.com/business). Approval is required to charge.
2. In the Affirm merchant dashboard, get your **Public** and **Private** API keys (sandbox first).

Turn it on

```
FLAG_AFFIRM_BNPL=1
AFFIRM_PUBLIC_API_KEY=...
AFFIRM_PRIVATE_API_KEY=...
AFFIRM_ENV=sandbox # switch to live after testing
PUBLIC_SITE_URL=https://yourdomain.com
```

Also flip the `affirm_bnpl` compliance flag on (Admin → Compliance Flags). Both the flag and the keys must be present, or financing stays hidden.

Flow (how the code works)

1. **Client** loads `Affirm.js` (<https://cdn1.affirm.com/js/v2/affirm.js>) with your public key.
2. On a real-goods listing, the buyer enters a shipping address and taps **Pay with Affirm**. The app calls `affirmCheckoutConfig` → returns the Affirm checkout object (items, total in cents, shipping, confirmation URL). Total includes your configured `STORE_MARKUP`.
3. The app calls `affirm.checkout(obj)` then `affirm.checkout.open()`; the buyer completes Affirm's flow and the client receives a **checkout_token**.
4. The app calls `affirmConfirm` with `{ listing_id, checkout_token }`. The server:
 - **authorizes** the charge (`POST /api/v2/charges`),
 - **atomically claims** the listing (active → sold) — if it lost the race, it **voids** the auth so the buyer isn't charged,
 - **captures** the charge (`POST /api/v2/charges/{id}/capture`) —

- you're paid upfront,
 - opens an **Order** (payment_method: "affirm", payment_captured: true) and kicks fulfillment (platform → aiOrderFulfillment; member seller → autoOrderFulfillmentAndFundsRelease).
5. Seller ships; funds/points release per the normal fulfillment lifecycle.

The real-goods guardrail (important)

affirmCheckoutConfig and affirmConfirm reject: affiliate listings (retailer fulfills), any listing with no real USD price, and anything whose category/type reads as points, store credit, gift card, virtual currency, or coins. Keep this guard — financing play/points credit is exactly what's not allowed.

Fees & economics

Affirm charges a merchant fee (typically ~a few percent + a flat amount, negotiated). You receive order_total - fee upfront. Build the fee into your marketplace margin, or note that Affirm purchases add the same STORE_MARKUP a card purchase does.

Test checklist (sandbox)

1. Keys + flag set; open a real-goods listing → "Pay with Affirm" appears.
2. Complete Affirm sandbox checkout → affirmConfirm authorizes + captures → Order created, payment_captured: true.
3. Try a points/affiliate/no-USD listing → financing is refused with a clear message.
4. Race test: two buyers → the loser's authorization is voided (not charged).
5. Switch AFFIRM_ENV=live only after your merchant account is approved for production.

Where it lives

- backend/functions/affirmCheckoutConfig/entry.ts — builds the checkout object (real-goods guard).
- backend/functions/affirmConfirm/entry.ts — authorize → claim → capture → order → fulfill.
- backend/sdk/feature-flags.ts — affirm_bnpl flag (off by default).
- Frontend: add the "Pay with Affirm" button + Affirm.js on eligible real-goods listings (calls the two functions above). Not yet wired into the marketplace UI — say the word and I'll add the button.